

Web Music Player

Domain Model

Nicolà Widmer · HSLU · Web Programming Lab · 2026-09-02

Contents

1	Purpose and Scope	2
2	Context Map	3
3	Shared Kernel — <code>src/shared/</code>	3
4	identity	4
	4.1 Aggregate User (root)	4
	4.2 Aggregate Session (root)	4
	4.3 Value objects, ports, use cases	4
5	songs	4
	5.1 Aggregate Song (root)	4
	5.2 Value objects	5
	5.3 Ports	5
	5.4 Use cases	5
	5.5 Events and HTTP surface	5
6	playlists	5
	6.1 Aggregate Playlist (root)	6
	6.2 Value objects, ports, use cases	6
7	Aggregates at a Glance	7
8	Domain Events	9
9	Invariants	9
10	Confirmed Design Decisions	10
11	Mapping to Code	10

1 Purpose and Scope

This document is the reference for the backend's domain layer: the bounded contexts, their aggregates, the value objects and invariants each aggregate guards, the ports the application layer depends on, and the one domain event that crosses a context boundary.

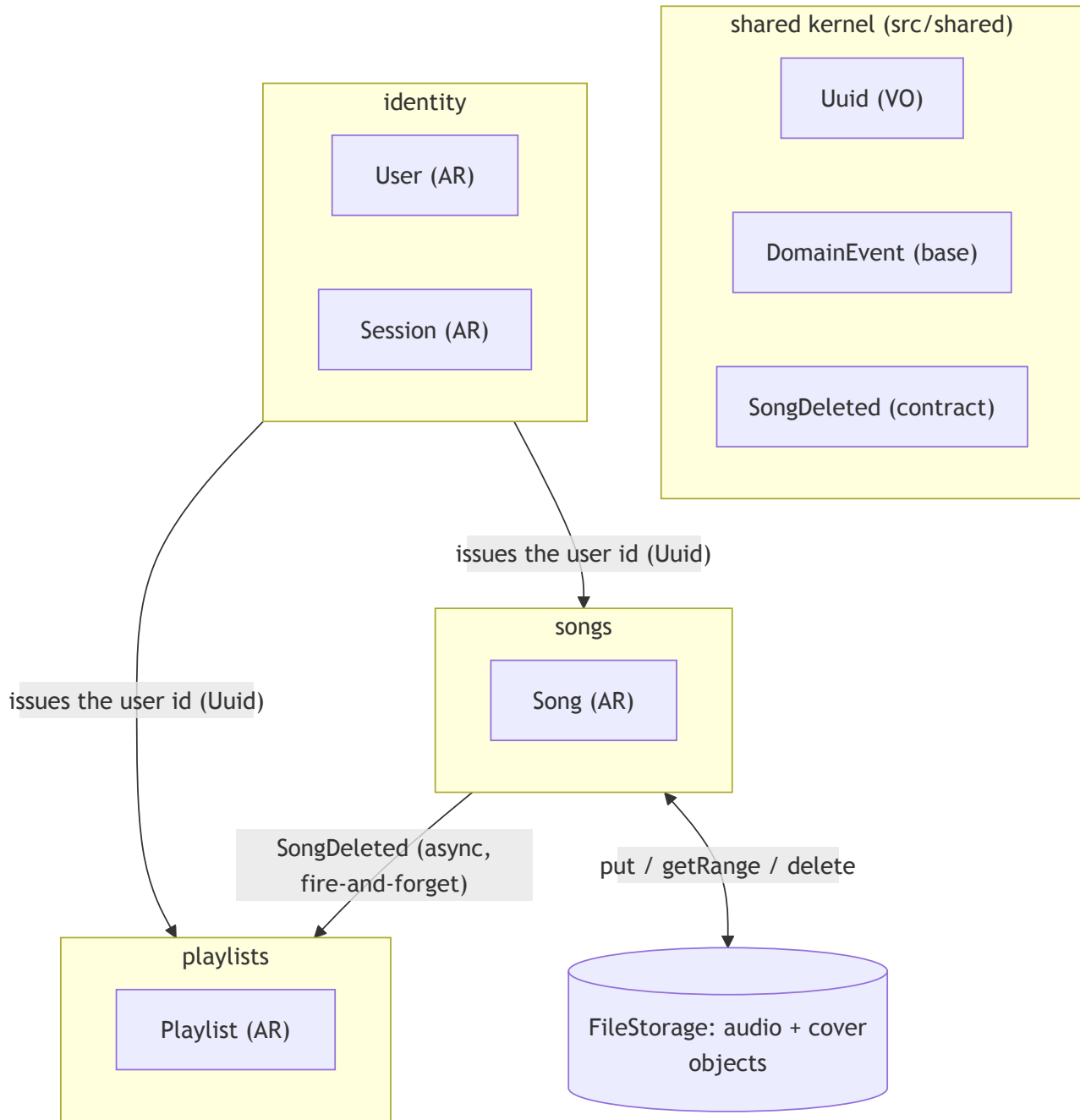
Three server-side bounded contexts, each a feature-folder Nest module (`0002-ddd-hexagonal-backend`):

Context	Responsibility
<code>identity</code>	Registration, sign-in, server-side sessions; the source of the authenticated user's id. Stories AUTH-1, AUTH-2.
<code>songs</code>	The music library: MP3 upload with validation and ID3 extraction, metadata CRUD, the audio- and cover-file lifecycle, and HTTP range streaming. Stories SNG-1, SNG-2, SNG-3, PB-1.
<code>playlists</code>	Named, ordered lists of songs; a song may appear more than once. Stories PL-1, PL-2, PL-3.

The proposal's glossary names a fourth context, `streaming`. It has no aggregate and no domain state — playback session state (current track, position, repeat, volume, resume) is **client** state in the Angular `player/` feature, persisted to `localStorage` (`0006-openapi-typed-client-tanstack-query`). Byte delivery with HTTP Range is therefore folded into `songs` as an application service, not a separate module.

Cross-context references are **by identity only**: a `Playlist` holds song ids (`Uuid`), never `Song` objects; `songs` and `playlists` hold the owner's id, never a `User`.

2 Context Map



SongDeleted is the **entire** server-side cross-context surface. Every other “X changed, view Y updates” in the proposal — a rename showing everywhere, a reorder driving auto-advance, the playing song being removed — is the frontend refetching, not a domain event.

3 Shared Kernel — src/shared/

Item	Role
Uuid (VO)	Validated v4 UUID. The type of every entity id and every cross-aggregate reference (ownerId, songId, entryId); minted by the IdGenerator port.
DomainEvent (base)	Marker + occurredAt for everything on the bus.
contracts/SongDeleted	The one published cross-context event. Extends DomainEvent; carries only ids.
Clock, IdGenerator	Port interfaces reused by more than one context.

4 identity

Upstream of everything; depends on no other context. Stories AUTH-1, AUTH-2.

4.1 Aggregate User (root)

Field	Type	Notes
id	Uuid	
email	Email (VO)	Normalised lower-case, format-valid. Uniqueness is a context rule checked by the Register use case, not by the aggregate.
passwordHash	PasswordHash (VO)	bcrypt output only. Plaintext is never stored and never held on the aggregate.
createdAt	Date	

- **Invariants:** valid Email; only a bcrypt hash is stored; minimum password length is enforced before hashing.
- **Behaviour:** `User.register(email, plaintext, hasher) (factory) · verifyPassword(plaintext, hasher): boolean`.

4.2 Aggregate Session (root)

Its own aggregate — a lifecycle distinct from **User**, revoked independently (0005-session-cookie-auth).

Field	Type	Notes
id	Uuid	Opaque v4 UUID; this is the cookie value.
userId	Uuid	
createdAt / expiresAt	Date	

- **Behaviour:** `Session.issue(userId, clock, ttl) · isValid(now): boolean`. Sign-out deletes the record.

4.3 Value objects, ports, use cases

Value objects	Email (normalise + validate) · PasswordHash (wraps the bcrypt string; never mixed with plaintext). The plaintext password is a plain string, min-length checked at the DTO; ids are the shared Uuid (§3).
Ports	<code>UserRepository</code> (save, byId, byEmail, existsByEmail) · <code>SessionRepository</code> (save, byId, deleteById, deleteExpired) · <code>PasswordHasher</code> · <code>Clock</code> · <code>TokenGenerator</code>
Use cases	<code>Register</code> · <code>SignIn</code> (verify → <code>Session.issue</code> → return token) · <code>SignOut</code> · <code>AuthenticateRequest(token) → Uuid</code> (the HTTP guard; with <code>AUTH_ENABLED=false</code> returns the fixed local user id and skips the cookie)
Events	None required.

5 songs

Owns the audio **and** cover file lifecycle and range streaming. Stories SNG-1, SNG-2, SNG-3, PB-1; quality goal 2.

5.1 Aggregate Song (root)

Field	Type	Notes
id	Uuid	
ownerId	Uuid	Every repository query is owner-scoped (0005-session-cookie-auth).
title	string	Required, non-empty (checked in the factory and the DTO). At creation, falls back to the filename when ID3 has no title.
artist / album	string?	Optional. Editable.
duration	int (seconds)	From ID3 or the decoder. Read-only after creation.

audio	AudioRef (VO)	Bundles { storageKey, sizeBytes, contentType = "audio/mpeg" } — three fields, not a string. Immutable — SNG-1: “the audio file is never modified”.
coverArt	CoverArtRef? (VO)	Bundles { storageKey, contentType, sizeBytes }; a separate FileStorage / S3 object, not a database blob (0004-metadata-postgres-blob-storage-port).
addedAt	Date	

- **Invariants:** title non-empty; duration and audio immutable; the upload is MP3 (MIME **and** extension) and at most 20 MB.
- **Behaviour:** Song.upload({ ownerId, id3, audioRef, coverRef? }) (factory — applies the title fallback, copies artist / album / duration from ID3) · editMetadata({ title, artist, album }) (title still required; duration, audio and cover untouched). No delete method — deletion is orchestrated by the use case.

5.2 Value objects

AudioRef · CoverArtRef (each bundles a storage key + size + content type) · Mp3Upload (transient: filename, bytes, declared MIME; isValidMp3(), withinSizeLimit()) · Id3Tags (transient: parsed title? / artist? / album? / durationSeconds? / coverBytes?).

title, artist, album are plain strings and duration a plain int — their only rule (title non-empty) lives in the factory and the DTO, not in a wrapper type.

5.3 Ports

SongRepository	save · byId(id, ownerId) · listByOwner(ownerId, sort ∈ {title, artist, dateAdded}) · remove(id)
FileStorage	put(key, stream, sizeBytes, contentType) · getRange(key, range?) → { stream, contentLength, totalSize, contentType } · delete(key) · stat(key). Local-FS default, S3 optional (0004-metadata-postgres-blob-storage-port). Handles both the audio object and the cover object.
Id3Reader	read(bytes) → Id3Tags
AudioProbe	duration(bytes) → seconds — fallback when ID3 has none.
Clock, IdGenerator	

5.4 Use cases

- UploadSong — validate → read ID3 → put the audio under a new opaque key → if a cover is present, put it under a second key → Song.upload → save.
- ListSongs(ownerId, sort) · GetSong(ownerId, id) · EditSongMetadata.
- DeleteSong — owner-scoped load → SongRepository.remove → FileStorage.delete(audio.storageKey) → FileStorage.delete(coverArt.storageKey) if any → publish SongDeleted. Ordered per 0004-metadata-postgres-blob-storage-port a failed object delete leaves an orphan for the reconciliation sweep.
- StreamSongAudio(ownerId, songId, rangeHeader) — owner-scoped resolve → parse Range → FileStorage.getRange → stream with Content-Range, Accept-Ranges: bytes, Content-Length, status 206 / 200; another owner’s song → 403 / 404.
- GetSongCover(ownerId, songId) — owner-scoped → FileStorage.getRange → 200 (cacheable), or 404 when there is no cover.

5.5 Events and HTTP surface

Publishes	SongDeleted(songId, ownerId, at) — the only one.
HTTP	GET /api/songs · GET /api/songs/:id · POST /api/songs (multipart) · PATCH /api/songs/:id · DELETE /api/songs/:id · GET /api/songs/:id/audio (Range) · GET /api/songs/:id/cover

6 playlists

Named, ordered lists. **A song may appear more than once**, so each entry carries its own local identity. Stories PL-1, PL-2, PL-3.

6.1 Aggregate Playlist (root)

Field	Type	Notes
id	Uuid	
ownerId	Uuid	
name	string	Required, non-empty (checked in the factory and the DTO). Duplicates are allowed across a user's playlists.
entries	PlaylistEntry[]	Ordered; local entities (below).
createdAt	Date	

6.1.1 Entity PlaylistEntry (inside the aggregate)

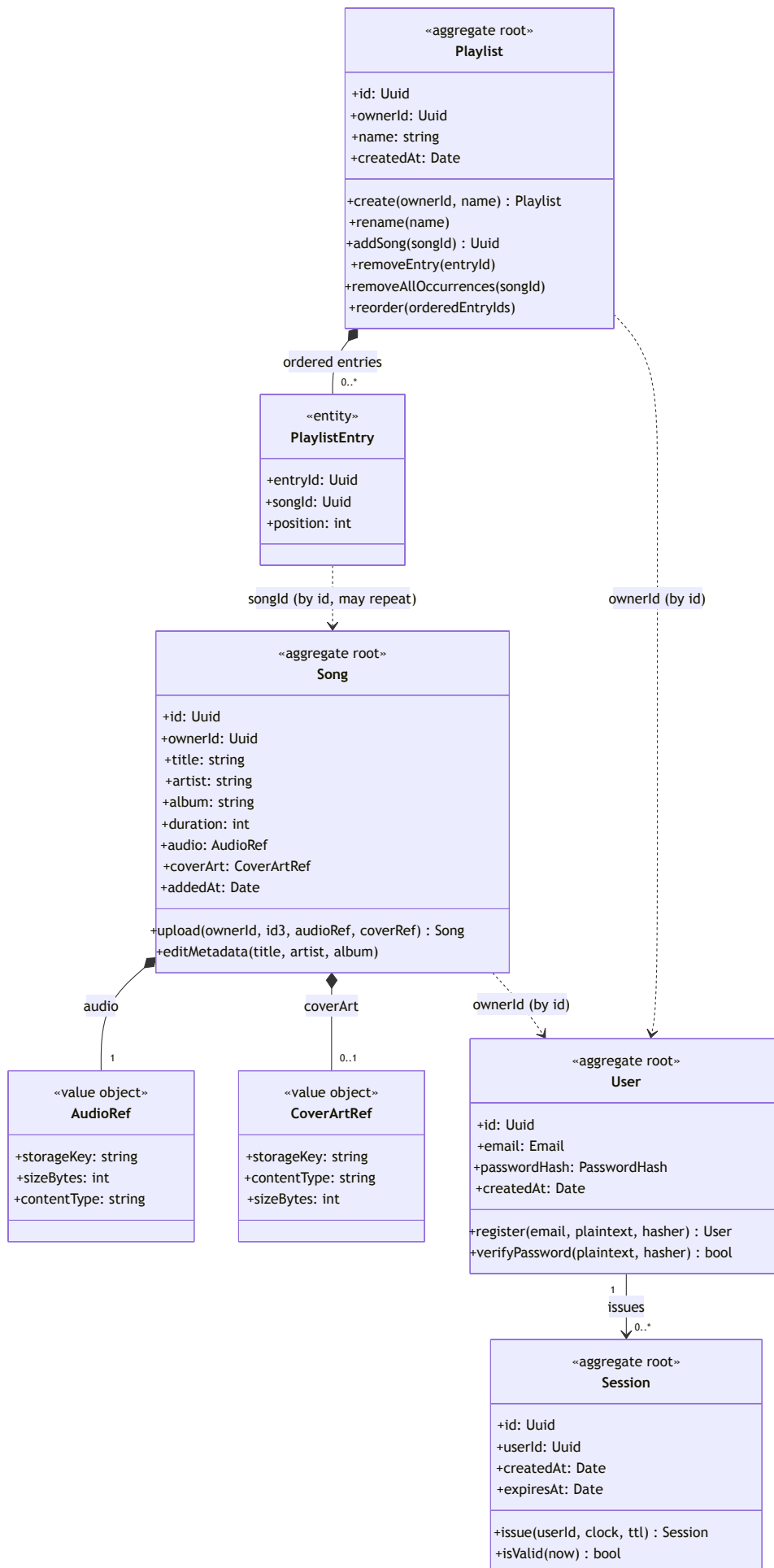
Field	Type	Notes
entryId	Uuid	Local identity; what add / remove / reorder address. Unique within the playlist (a v4 UUID is globally unique anyway).
songId	Uuid	By value. May repeat across entries.
position	int	Maintained contiguous 0 ... n-1.

- **Invariants (guarded by the root):** name non-empty; entryId unique within the playlist; position values contiguous and unique; a reorder argument is a permutation of the current entryId set. **No** uniqueness on songId.
- **Behaviour:**
 - Playlist.create(ownerId, name)
 - rename(name)
 - addSong(songId) — always appends a **new** PlaylistEntry with a fresh entryId at the end, even when that songId is already present.
 - removeEntry(entryId) — removes that one occurrence, re-packs position (PL-1: “affects only this playlist, remaining order preserved”).
 - removeAllOccurrences(songId) — drops every entry with that songId; used **only** by the SongDeleted handler.
 - reorder(orderedEntryIds) — permutation check, re-packs position (PL-3).
- The aggregate does **not** verify that a songId points at a real Song — that is a cross-context fact. A stale id (song deleted between the UI listing it and the add) is cleaned by SongDeleted plus the reconciliation sweep.

6.2 Value objects, ports, use cases

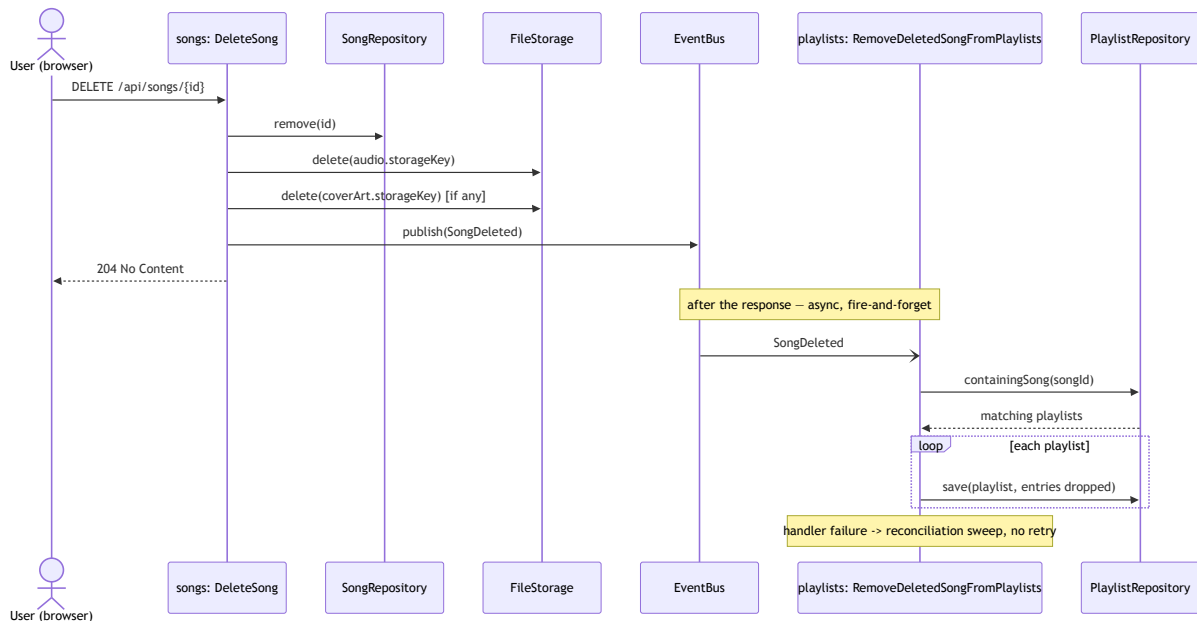
Value objects	None. name is a plain string; ids are the shared Uuid (§3).
Ports	PlaylistRepository — save · byId(id, ownerId) · listByOwner(ownerId) · remove(id) · containingSong(songId) (not owner-scoped: it serves the system event, not a user request)
Use cases	CreatePlaylist · ListPlaylists (name + track count) · GetPlaylist (ordered [{ entryId, songId, position }]) · RenamePlaylist · AddSongToPlaylist(ownerId, playlistId, songId) · RemovePlaylistEntry(ownerId, playlistId, entryId) · ReorderPlaylist(ownerId, playlistId, orderedEntryIds) · DeletePlaylist (the songs stay in the library) · handler RemoveDeletedSongFromPlaylists(SongDeleted) → for each containingSong(songId): removeAllOccurrences(songId) + save
Events	None published server-side.
HTTP	GET /api/playlists · POST /api/playlists · GET /api/playlists/:id · PATCH /api/playlists/:id(rename) · DELETE /api/playlists/:id · POST /api/playlists/:id/entries { songId } · DELETE /api/playlists/:id/entries/:entryId · PUT /api/playlists/:id/entries { entryIds: [...] } (reorder)

7 Aggregates at a Glance



artist, album and coverArt are optional. Dashed arrows (..>) are id-only references that cross a context boundary — no object graph, no import.

8 Domain Events



Event	SongDeleted(songId, ownerId, at)
Publisher	songs · DeleteSong
Subscribers	playlists · RemoveDeletedSongFromPlaylists; the frontend player (advance / stop the current track)
Transport	async, fire-and-forget, in-process (0002-ddd-hexagonal-backend)

9 Invariants

Context	Aggregate	Key invariants	Enforced by
identity	User	valid Email; only a bcrypt hash stored; minimum password length	User.register + Register (email uniqueness)
identity	Session	opaque UUID id; expiry respected; deletable	Session.issue / isValid; SignOut
songs	Song	title non-empty; duration + audio immutable; MP3-only, ≤ 20 MB	UploadSong (file rules) + Song.upload / editMetadata
playlists	Playlist	name non-empty (duplicates allowed); entryId unique in-playlist; position con-	Playlist root methods; owner scope in the use cases

		tiguous; reorder = permutation of <code>entryIds</code> ; <code>songId</code> may re- peat	
songs (streaming)	—	stateless; owner- scoped range reads	<code>StreamSongAudio</code>

10 Confirmed Design Decisions

Decision	Rationale
streaming folded into songs	No aggregate, no domain state; one developer should not maintain a projection plus sync handlers for a byte-serving endpoint. The glossary’s fourth context is a <code>songs</code> HTTP concern.
Session is its own aggregate	Lifecycle distinct from <code>User</code> ; revoked independently.
One <code>Uuid</code> type for every id	Entity ids and cross-aggregate references are all v4 UUIDs through one shared value object, not per-aggregate branded types (<code>SongId</code> , <code>PlaylistId</code> , ...). Less ceremony; the id’s shape stays explicit.
Value objects only where they earn it	<code>Email</code> , <code>PasswordHash</code> , <code>AudioRef</code> , <code>CoverArtRef</code> — they normalise, validate a format, or bundle several fields. <code>title</code> , <code>artist</code> , <code>album</code> , <code>playlist name</code> , <code>duration</code> are plain <code>string</code> / <code>int</code> ; the one rule (non-empty) sits in the aggregate factory and the DTO (<code>0006-openapi-typed-client-tanstack-query</code>).
A song may appear many times in one playlist	<code>PlaylistEntry</code> gets a local <code>entryId</code> ; add / remove / reorder address entries, not <code>songIds</code> . <code>removeAllOccurrences</code> exists only for the <code>SongDeleted</code> reaction.
playlists accepts <code>songIds</code> optimistically	No cross-context existence check. A dangling entry is cleaned by <code>SongDeleted</code> + the reconciliation sweep — consistent with the eventually-consistent event posture.
Cover art is a separate <code>FileStorage</code> / S3 object	Keeps PostgreSQL and its backups small (<code>0004-metadata-postgres-blob-storage-port</code>); <code>delete</code> removes the audio object and the cover object.
No server-side playback session	PB-2 / PB-3 / PB-4 persistence is <code>localStorage</code> in the Angular <code>player/</code> feature (<code>0006-openapi-typed-client-tanstack-query</code>); cross-device resume is out of scope (<code>arc42 §3</code>).

11 Mapping to Code

One feature folder per context under `backend/src/`, each split `domain/ · application/ · infrastructure/ · http/` + `<context>.module.ts` (`0002-ddd-hexagonal-backend`). Aggregates and value objects are plain classes in `domain/`; ports are abstract class DI tokens in `application/` bound to `infrastructure/` adapters in the module; use cases are `@Injectable()` services in `application/`; event handlers are `@EventHandler` classes in the subscribing context’s `application/`. The `eslint.config.mjs` boundary rule fails the build on a cross-context import or an inward-pointing violation.